

L2 - 13.2. Threat Hunting With YARA

Task 1 - Introduction

This room aims to demonstrate an active application of threat hunting with a specific focus on using YARA rules to hunt for Indicators of Compromise (IOC) related to malware. We will use a realistic scenario as the red wire throughout this room.

Task 2 - Scenario Description

Our threat hunting team is part of Belgium's national CSIRT. We work closely with the cyber threat intelligence and incident response teams to get ahead of the numerous threats targeting our constituents. One main category of our constituents is political parties. Our cyber threat intelligence team has picked up an interesting [article](#) from the Mandiant threat intelligence blog about a targeted cyber attack on a German political party. They have extracted all the relevant information and passed it to our team. We will analyze this information and look for opportunities to hunt the threat described in the article. Below is the information extracted by the cyber threat intelligence team. They have structured the information using the diamond model.

Threat Intelligence Provided: All the URLs and referenced files provided below are real and malicious. Do not open them outside of an isolated environment.

Adversary: APT29

Victim: German political party

Used Capabilities (TTPs)

ID	Technique
T1543.003 (opens in new tab).	Create or Modify System Process: Windows Service
T1012 (opens in new tab).	Query Registry
T1082 (opens in new tab).	System Information Discovery
T1134 (opens in new tab).	Access Token Manipulation

ID	Technique
T1057(opens in new tab)	Process Discovery
T1007(opens in new tab)	System Service Discovery
T1027(opens in new tab)	Obfuscated Files or Information
T1070.004(opens in new tab)	Indicator Removal: File Deletion
T1055.003(opens in new tab)	Process Injection: Thread Execution Hijacking
T1083(opens in new tab)	File and Directory Discovery

Answer the questions below

| What technique does ID T1134 describe?

Access Token Manipulation

| What does the detection rule M_APT_Dropper_Rootsaw_Obfuscated detect?

Detects obfuscated ROOTSAW payloads

Task 3 - Opportunities for Threat Hunting

This passage introduces threat hunting fundamentals as a prerequisite step for breaking down complex threat intelligence into an actionable threat hunt strategy.

Threat Hunting Styles: There are three styles of threat hunting: Structured hunting, unstructured hunting, and situational/entity-driven hunting.

Structured Hunting: Uses attacker TTPs and frameworks like MITRE ATT&CK to form hypotheses and detect threats early in the kill chain before damage occurs.

Unstructured Hunting: Uses technical Indicators of Compromise (IOCs)—such as file hashes or IP feeds—to run SIEM queries or YARA scans across historical data.

Combines structured hypothesis testing and unstructured IOC searches to protect critical assets (Crown Jewels) based on real-time changes in the threat landscape.

The primary sources of threat intelligence are threat reports from within the same business vertical and historical attacks.

Threat Hunting Process



Threat hunting consists of 3 phases:

1. **Trigger:** This is what initiates the threat hunt. This can be an IOC, a set of TTPs, a hypothesis, a system that behaves abnormally, articles on external blogs, reports by third parties, etc.
2. **Investigation:** A specific trigger is selected and used as a starting point for hunting activities. The threat hunter can use various tools to support the hunt for anomalies, such as YARA rules, Volatility, malware scanners, packet analyzers like Wireshark, and many more.
3. **Resolution:** If the threat hunter finds evidence of a breach, the incident response team is notified, and the incident response procedure is started.

Depending on the procedure, the threat hunter can support the IR team by scoping and digging deeper into the evidence found.

Opportunities: There are multiple opportunities provided within the received threat intelligence:

1. The received threat intelligence details specific TTPs attributed to APT29, which is known to target political entities.
 - This intelligence enables a *structured hunting style* using the TTPs included in the report to build a hypothesis.
2. The received threat intel includes Indicators of Compromise and YARA rules to hunt for malware.
 - This intelligence enables an *unstructured hunting style* using the IOCs provided.
3. The two opportunities above can be combined to enable a *situational or entity-driven hunting style*.

Answer the questions below

Which threat hunting style is proactive and uses indicators of attack and TTPs?
structured hunting

In which phase of the threat hunting process, tools like YARA or Volatility are used?
Investigation

You have received a threat intelligence report consisting only of Indicators of Compromise. What threat hunting style do you recommend to use?
unstructured hunting

Task 4 - YARA: Introduction

These YARA rules can be used to hunt for specific malware (in this scenario, the malware is WINELOADER). Before we get hands-on with these YARA rules, it is important to understand what YARA is. YARA stands for Yet Another Ridiculous Acronym. It is a tool Victor Alvarez of VirusTotal developed to assist malware

researchers in detecting and describing malware families. The main functionality of YARA is based on advanced pattern matching, explicitly tailored to malware. It can be best compared to using a supercharged grep with complex regular expressions in Linux. Just like the grep command, the YARA binary will iterate over all files in a designated path, trying to find a match with the information provided in the YARA rule. A YARA rule describes a malware family based on a pattern using a set of strings and Boolean logic.

Structure of a YARA Rule: A YARA rule uses descriptive language to define a pattern consisting of strings to match a Boolean condition specified at the end of the rule. The main parts of a YARA rule are the rule **name**, **meta**, **strings**, and **condition**. Below, we will discuss each part.

Rule Name: The **Rule name** is a descriptive name for the rule and starts with the keyword **rule**. Best practices include setting a name that clarifies what the rule is used for.

Meta: This part defines extra information like description, author, and more. Custom identifiers and value pairs can be freely created. The information defined in **meta** cannot be used in the **condition** part. Whether to include this part or not is entirely up to you. The rule will work completely fine without it. It is, however, recommended to include the *meta* part with some basic information, including the author and the description of what to use the rule for.

Strings: In this part of the rule, matching strings are defined. Multiple types of strings can be defined, which is essential for creating functional rules.

Condition: In this part of the rule, a matching condition is defined using the identifiers defined in the **strings** part.

Example of a YARA Rule: Below is an example of a YARA rule we received from the CTI team. In this example, all four parts discussed in the previous paragraph are present:

1. **Rule name:** M_APT_Dropper_Rootsaw_Obfuscated. The rule's title is well-chosen and gives the user a good idea of what to use it for. In this case, it is to detect a dropper called Rootsaw that is obfuscated.
2. **Meta:** It is good practice to include relevant data that provides more information about the rule. This helps the user of the YARA rule know what to use the rule for, who wrote it, and where to apply it.

3. **Strings:** The strings included in this example help the user find a file containing those strings. How do malware analysts choose those strings? They analyze the malware and determine what uniquely identifies it. The strings used are *text strings*. The first two lines are straightforward.
4. **Condition:** This rule requires that all defined strings be present to have a match. This means all the strings defined in part 3 must have a match in the same file being matched against.

```
rule M_APT_Dropper_Rootsaw_Obfuscated 1
{
  meta: 2
    author = "Mandiant"
    disclaimer = "This rule is meant for hunting and is not tested to run in
a production environment."
    description = "Detects obfuscated ROOTSAW payloads"

  strings: 3
    $ = "function _"
    $ = "new XMLHttpRequest();"
    $ = "'\\x2e\\x7a\\x69\\x70'"
    $ = "'\\x4f\\x70\\x65\\x6e'"
    $ = "\\x43\\x3a\\x5c\\x57"

  condition: 4
    all of them
}
```

Only two parts are required for a rule to function: the rule name and the condition. All the other parts are optional. However, adding strings to a rule is recommended if you want to create complex, functional YARA rules.

Answer the questions below

┃ Apart from the rule name, which other section is also required in a YARA rule?

condition

Task 5 - YARA: Strings and Conditions

False Positives: An essential part of threat hunting is excluding false positives. When we look at YARA, this means creating rules that uniquely identify the threat you are looking for. This is easier said than done. YARA rules can get complex

very fast when tailored toward specific malware. Using well-written and tested YARA rules is a must. This leaves us with two choices: You either dive deep into the specifics of writing YARA rules or use YARA rules created by experts (like those included in the threat intelligence we received in Task 3). Of course, the combination of these two options is also viable. Even when just using pre-created YARA rules, it is crucial that you understand them.

Strings: In this paragraph, we will investigate what strings we can use within a YARA rule. We will discuss the general types of available strings without going too deep in each category. Most modifier keywords we will encounter throughout this paragraph can be combined. It is out of scope to discuss all the possible combinations.

Text Strings: In its simplest form, we can define an ASCII-encoded string that matches some text we seek. It is essential to mention that the specified string is case-sensitive. Let us look at an example:

```
rule textString
{
    strings:
        $1 = "This is an ASCII-encoded string" //strings
are defined between double quotes
        $2 = "This is an ascii-encoded string" //not the
same as $1.

    condition:
        all of them
}
```

Wide-Character Strings: Strings can be encoded in different ways. One way often found in binaries is encoding the string as two bytes per character instead of the traditional one-byte ASCII encoding.

Hexadecimal Strings: When malware analysts start analyzing malware, they often use a disassembler and debugger like IDA Pro to dismantle binaries. Often, the pieces of code they uncover are displayed in hexadecimal. We can then use the hexadecimal strings uncovered during analysis to create our own YARA rules. Sequences of hexadecimal characters are often more challenging for attackers to

obfuscate and hide. So, these hexadecimal strings provide an excellent opportunity to uniquely identify a certain malicious binary. Let us look at an example of hexadecimal strings in a YARA rule.

```
rule hexString
{
    strings:
        $1 = { E2 34 B6 C8 A3 FB } // Hexadecimal strings
are defined between {}

    condition:
        $1
}
```

XOR Strings: Malware creators often use XOR to encrypt their code, making it harder for malware analysts to analyze. It also helps evade anti-virus signatures. The XOR string support in YARA helps us to hunt for XOR encrypted string variations with 1-byte keys. Let us look at an example:

```
rule xorString
{
    strings:
        $1 = "http://maliciousurl.thm" xor // This line w
ill look for all variations possible with a 1-byte XOR key

    condition:
        $1
}
```

Malware authors will often use encoding to evade detection. One encoding technique that is often used is base64. YARA supports looking for base64 encoded strings. To do this, you can use the modifier *base64* after defining the string. YARA will search for the base64 encoded string when running the YARA rule. Let us have a look at an example:

```

rule base64String
{
    strings:
        $1 = "This is a regular string" base64 // At
runtime YARA will encode the string with base64 and look for
matches.

    condition:
        $1
}

```

Regular Expressions: Just like with the grep command in Linux, using regular expressions makes YARA powerful. You can define regular expressions the same way as strings, with the only difference being forward slashes instead of double quotes. A bonus is that the above modifiers can also be applied to these regular expressions.

```

rule regularExpression
{
    strings:
        $1 = /THM\[a-zA-Z]{3}\]/ // This regex will
match any string that starts with "THM{" , ends with "}" and h
as 3 alphabetic characters (lower-case or upper-case) between
the curly brackets.

    condition:
        $1
}

```

Conditions: Once you have defined your strings, it is crucial to define how to combine them and match the files you are searching for. YARA offers great flexibility when making different combinations. YARA includes Boolean, relational, arithmetic, and bitwise operators. Additionally, some keywords can be used. The table below shows an overview of the operators and keywords:

Boolean operators	Relational operators	Arithmetic operators	Bitwise operators	Keywords
and	>=	+	&	1 of them
or	<=	-		any of them
not	<	*	<<	none of them
	>	\	>>	contains
	==	%	~	icontains
	!=		^	startswith
				startswith
				endswith
				iendswith
				iequals
				matches
				not defined
				filesize

Answer the questions below

What modifier should be used if you want to search for 2-byte encoded characters?

wide

Which condition matches only when none of the defined strings are present?

none of them

Task 6 - Environment and Setup

You can start the machine by clicking the [Start Lab Machine](#) button below. The VM will take approximately 2 minutes to boot up, and will start in split view. If the VM is not visible, use the blue [Show Split View](#) button at the top of the page. All the necessary files used throughout this room are located in [C:\TMP\](#).

Task 7 - YARA: How To Use YARA Rules To Hunt for Indicators of Compromise

During this task, we will learn how to use a YARA rule to hunt for Indicators of Compromise for the WINELOADER malware. It is important to note that YARA rules can be run standalone or as part of a security product like Kaspersky, VirusTotal, Trend Micro, and more.

Basic Syntax for YARA: Open up a PowerShell window as administrator and enter the command `yara64`, where you should get the following output:

```
PS C:\TMP> yara64
yara: wrong number of arguments
Usage: yara [OPTION]... [NAMESPACE:]RULES_FILE... FILE | DIR
| PID
```

Try `--help` for more options

Enter the following command to show the help page and see the available options: `yara64 --help`. The help page is a great reference to all the different arguments available. The table below lists a few of the common flags used.

Short Flag	Long Flag	Description
-r	--recursive	Scan directories recursively
-n	--negate	Print only rules that weren't matched
-S	--print-stats	Print metadata related to the performance and efficiency of the rule
-s	--print-strings	Print the strings that were matched in a file
-X	--print-xor-key	Print xor key and plaintext of matched strings
-v	--version	Show the YARA version
-p	--threads=N	Use N threads to scan a directory

Run a YARA Rule for the First Time: We will use a basic YARA rule we wrote for this hands-on. Let's examine the rule.

Administrator: Windows PowerShell

```
PS C:\TMP> get-content C:\TMP\YARARULES\myfirstrule.yar
rule myfirstrule {
```

```

meta:
    Description = "Searches for the string tryhackme"
    Author = "TryHackMe"

strings:
    $s = "tryhackme"

condition:
    $s
}

```

This rule searches for the string `tryhackme` in the `C:\TMP\` directory. Enter the following command: `yara64 C:\TMP\YARARULES\myfirstrule.yar C:\TMP\` to start searching using `myfirstrule.yar`.

The result of running this rule should be as follows:

Administrator: Windows PowerShell

```

PS C:\TMP> yara64 C:\TMP\YARARULES\myfirstrule.yar C:\TMP\
myfirstrule C:\TMP\test.txt

```

- The used command is shown on the first line.
- The second line shows a match for a file named `test.txt`. Open this file and verify the result.
- You can use the `myfirstrule.yar` as a starting point for writing your own YARA rules.

Hunt for WINELOADER Malware Indicators of Compromise: For this walkthrough, we will use the YARA rules that the CTI team provided. A malware sample of WINELOADER is included in the `C:\TMP\` folder. Open a PowerShell administrator window and enter the following command to hunt for WINELOADER malware:

```
yara64 C:\TMP\YARARULES\WINELOADER1.yar C:\TMP\
```

The following result should be displayed:

Administrator: Windows PowerShell

```
PS C:\TMP> yara64 C:\TMP\YARARULES\WINELOADER1.yar C:\TMP\
M_APT_Download_WINELOADER_1 C:\TMP\72b92683052e0c813890caf
7b4f8bfd331a8b2afc324dd545d46138f677178c4.exe
```

- The first line is the command.
- The second line shows a match, indicating that a sample of WINELOADER has been found.
- Now do the same for the other rules `WINELOADER2.yar` and `ROOTSAW.yar`

Combine Multiple Rules in One File: Rules can also be combined in one file.

Combining the three rules in one file could be interesting in our scenario. There are no specific guidelines on when to combine rules in one file. One way could be to group rules that hunt for the same malware family:

- Create a new file in `C:\TMP\YARARULES\` and name it `WINELOADERCOMBO.yar`.
- Copy the content of `WINELOADER1.yar`, `WINELOADER2.yar`, and `ROOTSAW.yar` in the newly created file. Leave a space between each rule for readability. The result should look something like this:

Administrator: Windows PowerShell

- Open a PowerShell administrator window and enter the following command to hunt for WINELOADER: `yara64 C:\TMP\YARARULES\WINELOADERCOMBO.yar C:\TMP\`
- The same results should be displayed as running each file separately.

Answer the questions below

| What option do you need to pass to ensure you scan all directories recursively?

-r

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\analyst> cd C:\TMP\YARARULES\
PS C:\TMP\YARARULES> dir

Directory: C:\TMP\YARARULES

Mode                LastWriteTime         Length Name
----                -
-----            8/29/2024   7:00 AM             367 checkregistry.yar
-a-----          9/10/2024   2:52 PM             211 myfirstrule.yar
-----            8/29/2024   7:00 AM             568 ROOTSAW.yar
-----            8/29/2024   7:00 AM             500 Wineloder.yar
-----            8/29/2024   7:00 AM             507 WINELOADER1.yar
-----            8/29/2024   7:00 AM             816 WINELOADER2.yar

PS C:\TMP\YARARULES> yara64.exe .\myfirstrule.yar C:\TMP\
myfirstrule C:\TMP\test.txt
PS C:\TMP\YARARULES> yara64.exe -s .\myfirstrule.yar C:\TMP\
myfirstrule C:\TMP\test.txt
0x0:$s: tryhackme
PS C:\TMP\YARARULES> yara64.exe .\WINEOADER1.yar C:\TMP\
error: could not open file: .\WINEOADER1.yar
PS C:\TMP\YARARULES> yara64.exe .\WINELOADER1.yar C:\TMP\
M_APT_Downloader_WINELOADER_1 C:\TMP\72b92683052e0c813890caf7b4f8bfd331a8b2afc324dd545d46138f677178c4.exe
PS C:\TMP\YARARULES>
```

Task 8 - Indicators of Compromise Detected - Now What

This task will focus on what happens after discovering a confirmed Indicator of Compromise. After discovering a true positive Indicator of Compromise on a system, the first thing you should do is detailed in the incident response procedure. Any company serious about security will have an incident response policy that explains all the steps to follow before, during, and after an incident.

Answer the questions below

| What does DAIR stand for?

Dynamic Approach to Incident Response

Task 9 - YARA: Hands-on Exercise

The following exercises combine the rule-writing concepts from the previous tasks, including string modifiers, conditions, and recursive scanning.

Exercise 1: Write a YARA rule to find the file that contains **the pattern** `THM{}`. Use the `C:\TMP\Exercisel\` path as the target in the YARA command, enter the flag as the answer.

Exercise 2: Write a YARA rule that finds the file that contains the following strings: `Yet another`, `Ridiculous acronym`. Use the `C:\TMP\Exercise2\` path as the target in the YARA command. Enter the name of the file as the answer.

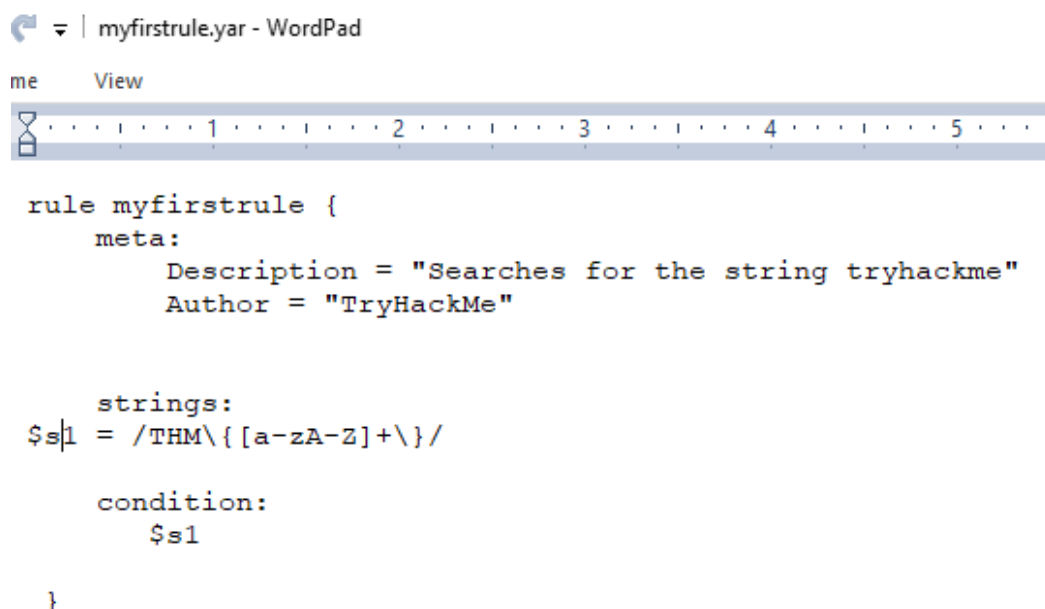
Exercise 3: Write a YARA rule that searches for the file that contains the base64 encoded string `THM{This was a really fun exercise}`. Use the `C:\TMP\Exercise3\` path as the target in the YARA command, and enter name of the file as the answer.

Exercise 4: Write a YARA rule that searches for the XOR encrypted string `THM{FoundSomethingHidden}` in the `C:\TMP` directory and subdirectories. Fill in the encrypted text and XOR key used.

Answer the questions below

What is the flag found in exercise 1?

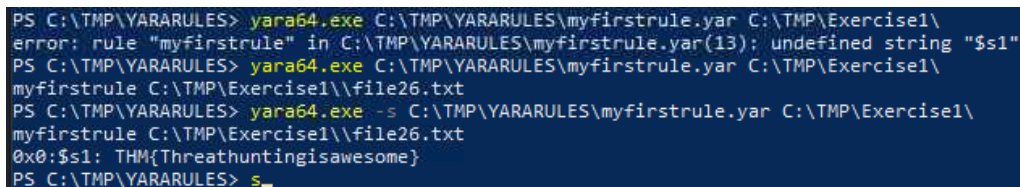
THM{Threathuntingisawesome}



```
rule myfirstrule {
    meta:
        Description = "Searches for the string tryhackme"
        Author = "TryHackMe"

    strings:
        $s1 = /THM\[a-zA-Z]+\]/

    condition:
        $s1
}
```

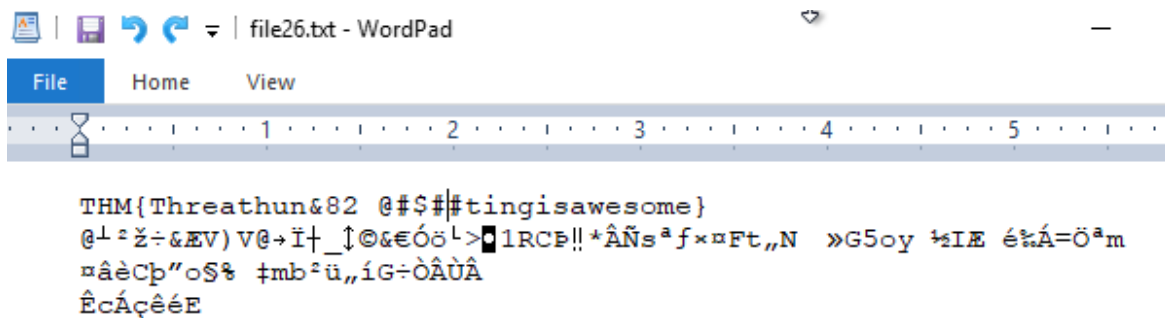


```
PS C:\TMP\YARARULES> yara64.exe C:\TMP\YARARULES\myfirstrule.yar C:\TMP\Exercise1\
error: rule "myfirstrule" in C:\TMP\YARARULES\myfirstrule.yar(13): undefined string "$s1"
PS C:\TMP\YARARULES> yara64.exe C:\TMP\YARARULES\myfirstrule.yar C:\TMP\Exercise1\
myfirstrule C:\TMP\Exercise1\file26.txt
PS C:\TMP\YARARULES> yara64.exe -s C:\TMP\YARARULES\myfirstrule.yar C:\TMP\Exercise1\
myfirstrule C:\TMP\Exercise1\file26.txt
0x0:$s1: THM{Threathuntingisawesome}
PS C:\TMP\YARARULES> s
```

What is the filename found in exercise 2?

file10.txt

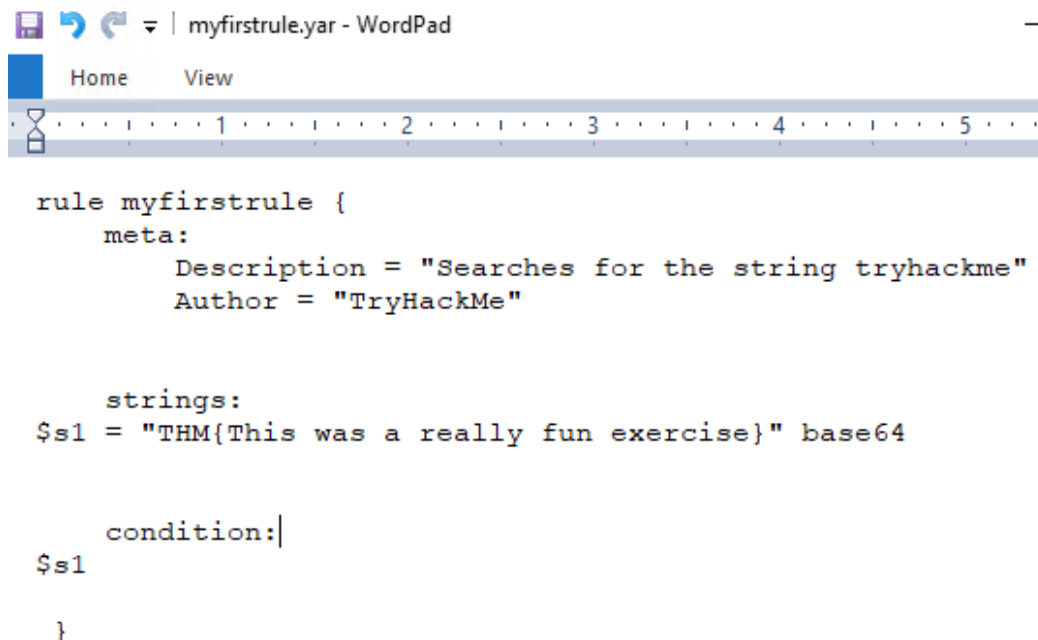
```
myfirstrule C:\TMP\Exercise1\\file26.txt
0x0:$s1: THM{Threathuntingisawesome}
PS C:\TMP\YARARULES> yara64.exe -s C:\TMP\YARARULES\myfirstrule.yar C:\TMP\Exercise1\
PS C:\TMP\YARARULES> yara64.exe -s C:\TMP\YARARULES\myfirstrule.yar C:\TMP\Exercise2\
myfirstrule C:\TMP\Exercise2\\file10.txt
0x11fee:$s1: Y\x00e\x00t\x00 \x00a\x00n\x00o\x00t\x00h\x00e\x00r\x00
0x1f038:$s2: R\x00i\x00d\x00i\x00c\x00u\x00l\x00o\x00u\x00s\x00 \x00a\x00c\x00r\x00o\x00n\x00y\x00m\x00
PS C:\TMP\YARARULES>
```



What is the filename found in exercise 3?

file13.txt

```
0x7da51:$s1: VEhNe1RoAXMgd2FzIGEgcmVhbGx5IGZ1biBleGVyY2l2ZX
0x7c251:$s1: VEhNe1RoAXMgd2FzIGEgcmVhbGx5IGZ1biBleGVyY2l2ZX
0x7ca51:$s1: VEhNe1RoAXMgd2FzIGEgcmVhbGx5IGZ1biBleGVyY2l2ZX
0x7d251:$s1: VEhNe1RoAXMgd2FzIGEgcmVhbGx5IGZ1biBleGVyY2l2ZX
0x7da51:$s1: VEhNe1RoAXMgd2FzIGEgcmVhbGx5IGZ1biBleGVyY2l2ZX
0x7e251:$s1: VEhNe1RoAXMgd2FzIGEgcmVhbGx5IGZ1biBleGVyY2l2ZX
0x7ea51:$s1: VEhNe1RoAXMgd2FzIGEgcmVhbGx5IGZ1biBleGVyY2l2ZX
0x7f251:$s1: VEhNe1RoAXMgd2FzIGEgcmVhbGx5IGZ1biBleGVyY2l2ZX
0x7fa51:$s1: VEhNe1RoAXMgd2FzIGEgcmVhbGx5IGZ1biBleGVyY2l2ZX
PS C:\TMP\YARARULES> yara64.exe C:\TMP\YARARULES\myfirstrule.yar C:\TMP\Exercise3\
myfirstrule C:\TMP\Exercise3\\file13.txt
PS C:\TMP\YARARULES>
```



```
rule myfirstrule {
    meta:
        Description = "Searches for the string tryhackme"
        Author = "TryHackMe"

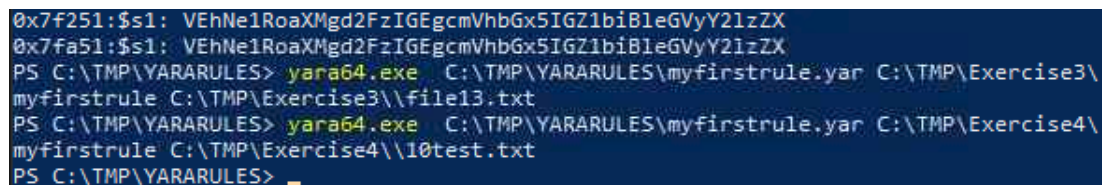
    strings:
        $s1 = "THM{This was a really fun exercise}" base64

    condition:
        $s1

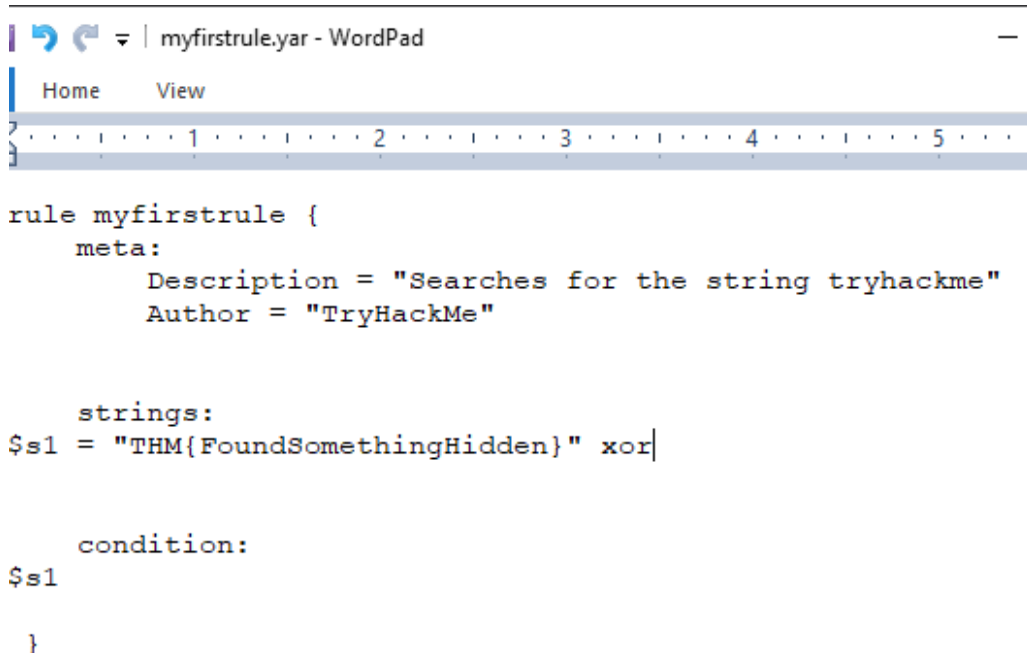
}
```

What was the XOR key used for encryption in exercise 4?

0x01



```
0x7f251:$s1: VEhNe1RoaxMgd2FzIGEgcmVhbGx5IGZ1biBleGVyY2lzMZx
0x7fa51:$s1: VEhNe1RoaxMgd2FzIGEgcmVhbGx5IGZ1biBleGVyY2lzMZx
PS C:\TMP\YARARULES> yara64.exe C:\TMP\YARARULES\myfirstrule.yar C:\TMP\Exercise3\
myfirstrule C:\TMP\Exercise3\file13.txt
PS C:\TMP\YARARULES> yara64.exe C:\TMP\YARARULES\myfirstrule.yar C:\TMP\Exercise4\
myfirstrule C:\TMP\Exercise4\10test.txt
PS C:\TMP\YARARULES>
```



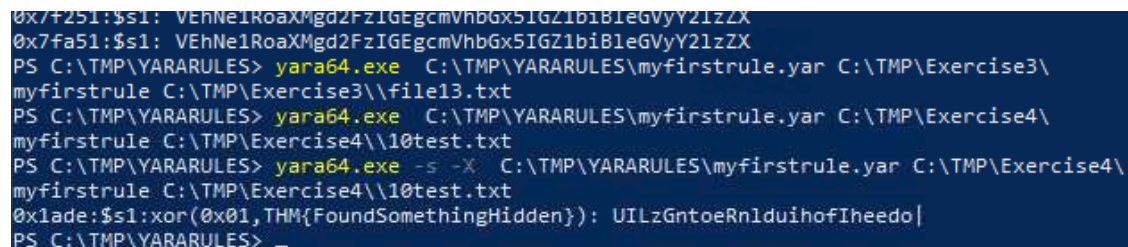
```
rule myfirstrule {
    meta:
        Description = "Searches for the string tryhackme"
        Author = "TryHackMe"

    strings:
        $s1 = "THM{FoundSomethingHidden}" xor|

    condition:
        $s1
}
```

What encrypted string did you find in exercise 4?

UILzGntoeRnlduihofIheedo|



```
0x7f251:$s1: VEhNe1RoaxMgd2FzIGegcmVhbGx5IGZ1biBleGVyY2lZZX
0x7fa51:$s1: VEhNe1RoaxMgd2FzIGegcmVhbGx5IGZ1biBleGVyY2lZZX
PS C:\TMP\YARARULES> yara64.exe C:\TMP\YARARULES\myfirstrule.yar C:\TMP\Exercise3\
myfirstrule C:\TMP\Exercise3\file13.txt
PS C:\TMP\YARARULES> yara64.exe C:\TMP\YARARULES\myfirstrule.yar C:\TMP\Exercise4\
myfirstrule C:\TMP\Exercise4\10test.txt
PS C:\TMP\YARARULES> yara64.exe -s -X C:\TMP\YARARULES\myfirstrule.yar C:\TMP\Exercise4\
myfirstrule C:\TMP\Exercise4\10test.txt
0x1ade:$s1:xor(0x01,THM{FoundSomethingHidden}): UILzGntoeRnlduihofIheedo|
PS C:\TMP\YARARULES>
```

Task 10 - Conclusion

We started this room with a realistic scenario that provided us with a generous load of threat intelligence. We then used this intelligence to mount a threat hunt. It became clear that a threat hunt is not just some random activity. It is a well-thought-out activity with multiple threat styles and a process that can be used. We focused on a structured hunting style using the provided IOCs. We then chose YARA as a tool to hunt for malware. We learned the capabilities of YARA and how to use YARA rules for hunting. We also wrote a basic YARA rule to conduct a simple threat hunt.